Building Web and Mobile Apps WS2025/26 (Prof. Dr. Cathrin Möller)

Exercise 2 - 3.11./4.11.2025

Practical Exercises

Do not hesitate to ask questions or if you want feedback for your coding style.

Please commit and push to GitLab regularly and ensure @*cathrin.moeller* and my two tutors @pinky-jitendra.tinani and @rahul.patil are invited as a reporter.

Learning goals for today
Today we want to work with **Typescript**, which is the most common task in frontend development. We will design an interface, which is crucial to match typed objects in the API of any backend.
We also want to get started using the **Node expressJS** framework to give us a backend that we can talk to from our frontend. We will do requests from a HTML form and via Javascript using the **fetch API**. We also want to practice mapping and parsing data by using TS **interfaces** and ES6 array functions. Finally, we want to load **JSON** content from a local file.

1) Create a new package (folder) named „user“. Design a **typescript interface** for **user** data including id, name, a full postal address, eMail and telephone number and add it in a TS file. Separate the user and the address interface.
   Create a new const variable with an example instance of a user.

2) To test the file, you can load the transpiled JS as an externalized script from a HTML file (e.g. index.html within /user which is then accessible via localhost:8080/user) and use console.log() of the created user within the TS code to be able to debug that it is working once running in the browser. **Transpile** the TS to JS. You might add a shortcut script to package.json for that.
Info: We will add templating in later exercises which allows to display the content as HTML.

3) Add a hardcoded **JSON** file *users.json* that contains multiple (3+) arbitrary users with addresses which match the defined interfaces.
Add a typescript file or extend the existing one (with the **interfaces**) to define **classes** *User* and *Address* that implement the interfaces and which contain a constructor that allows to create an entity of User or Address easily.
Now we need an **async** typescript function *loadJsonFile()* that loads the content from that hardcoded JSON file using **await** and the **fetch** API (the path can be your local file path).

Hint: In your IDE a Typesript warning might appear that Promises are no built-in ES5 feature. If you compile the TS code to JS using the command line command with flags „**tsc script.ts --lib dom,es2015**“, everything will work just fine and you can ignore the warning.

You can parse the received content from the fetch API to JSON using **.json()**.
Now, map the content of the json data to class instances of the typed interface s (using **.map()**). You will need to call the constructor of User and Address and inject the fields that you retrieve calling map().

Hint: use console.log() or other debugging frequently to ensure the parsed content is as expected. You can execute your Typescript by loading the compiled JS from a served HTML file and calling the defined function inside the TS code.
!Ensure your caching is deactivated, otherwise you might see outdated JS content.

4) Make the address **optional** and all user fields **mandatory** in the user **interface**.
Hint: An attribute can be marked as optional with **?** after the attribute name. It is then skipped if the value for an attribute is missing - instead of being undefined.

For mandatory fields you can define **default values in the constructor** to ensure that a field gets e.g. the value of an empty string instead of being undefined.

Add additional incomplete users (no email, no phone, no address) to the JSON file and map them.
There should be no „undefined“ in the value for any attribute.

Compile the TS code to JS. Run it and print the mapped content to the browser console using console.log().

5) **Node Express JS Hello World**
Create a new backend application in any separate folder (backend) by creating an (empty) app.js file.
Running „npm init“ you can create a package.json for this additional project.
Running „npm install express --save“ you can add express to your dependencies.
How does the package.json look like?

Add some ExpressJS code to your app.js with a GET mapping to the path „/„. Return „HelloWorld“ as a response. Configure your backend to start using port 8081 in order not to conflict with a frontend running already at port 8080.

Run the backend via „node app.js“ from the command line (or a start script in your package.json).

Visit localhost:8081 if your app listens to port 8081 and see HelloWorld.

6) **Return users as JSON from the backend**
Add a new GET endpoint to your backend for the path „/users“ and return the JSON content with hardcoded userdata from the file you already created in your frontend code. You can copy paste the file to your backend folder.
Loading a JSON file in NodeJS is simple using ***require*** (check the lecture slides) (the argument is a string of the complete relative filepath of the JSON file).